Học phần: Trí Tuệ Nhân Tạo Nâng Cao



BÁO CÁO
Lab03: Tìm kiếm cục bộ

Giảng viên hướng dẫn: Đỗ Như Tài

Nhóm: 03
Sinh viên thực hiện:
Trương Chí Long - MSSV: 3123410198

TP. Hồ Chí Minh, Năm 2026

MỤC LỤC

Đề mục / Nội dung báo cáo	Phần / Mục
PHẦN 1. TỔNG QUAN LÝ THUYẾT VÀ MỤC TIÊU BÀI THỰC HÀNH	Mục 1
1.1. Bản chất của Tìm kiếm cục bộ (Local Search)	Mục 1.1
1.2. Tổng hợp các thuật toán Tìm kiếm cục bộ khảo sát trong Lab 03	Mục 1.2
PHẦN 2. NỘI DUNG ĐÃ THỰC HIỆN VÀ KẾT QUẢ THỰC NGHIỆM (BÀI 1 - BÀI 8)	Mục 2
2.1. Bài 1: Tìm kiếm cục bộ trên mê cung - Simple Hill Climbing	Mục 2.1
2.2. Bài 2: Mô hình hóa bài toán n Quân hậu (n-Queens)	Mục 2.2
2.3. Bài 3: Steepest-ascent và Stochastic Hill Climbing (n-Queens)	Mục 2.3
2.4. Bài 4: Random Restart và Simulated Annealing (n-Queens)	Mục 2.4
2.5. Bài 5: Khảo sát lịch giảm nhiệt độ (Cooling Schedules)	Mục 2.5
2.6. Bài 6: Tối ưu hàm đếm xung đột O(n) và biên dịch JIT bằng Numba	Mục 2.6
2.7. Bài 7: Tìm kiếm cục bộ cho bài toán Người du lịch (TSP)	Mục 2.7
2.8. Bài 8: Phân tích hội tụ, khả năng mở rộng và toán tử di chuyển mới	Mục 2.8
PHẦN 3. BẢNG TỔNG HỢP KẾT QUẢ THÍ NGHIỆM	Mục 3
PHẦN 4. NHẬN XÉT CHUYÊN SÂU & BÀI HỌC RÚT RA QUA BÀI TẬP	Mục 4
4.1. Hiện tượng cực trị cục bộ (Local Optimum, Plateau, Ridge)	Mục 4.1
4.2. Cơ chế vượt bẫy của Random Restart và Simulated Annealing	Mục 4.2
4.3. Đánh đổi chất lượng lời giải, thời gian chạy và khả năng mở rộng	Mục 4.3
4.4. Hạn chế của các thuật toán tìm kiếm cục bộ	Mục 4.4
PHẦN 5. TRẢ LỜI 10 CÂU HỎI THẢO LUẬN	Mục 5
PHẦN 6. KẾT LUẬN	Mục 6

PHẦN 1. TỔNG QUAN LÝ THUYẾT VÀ MỤC TIÊU BÀI THỰC HÀNH

1.1. Bản chất của Tìm kiếm cục bộ (Local Search)

Trong nhiều bài toán tối ưu hóa tổ hợp và thỏa mãn ràng buộc của Trí tuệ nhân tạo (như xếp lịch, bố trí vi mạch, n-Queens hay bài toán Người du lịch - TSP), đường đi từ trạng thái khởi đầu đến trạng thái đích không mang ý nghĩa ứng dụng; điều duy nhất ta quan tâm là cấu hình trạng thái cuối cùng. Các thuật toán Tìm kiếm cục bộ (Local Search) hoạt động bằng cách duy trì một trạng thái hiện tại duy nhất và liên tục dịch chuyển sang các trạng thái lân cận (neighbors) nhằm cải thiện giá trị của hàm mục tiêu (Objective Function / Heuristic Cost).

- **Ưu điểm vượt trội về bộ nhớ:** Vì không lưu trữ cây tìm kiếm (Frontier và Explored Set) như BFS hay A*, tìm kiếm cục bộ chỉ sử dụng không gian bộ nhớ hằng số O(1) (hoặc O(n) để lưu 1 trạng thái), cho phép áp dụng trên các không gian trạng thái khổng lồ.
- **Thách thức lớn nhất - Cực trị cục bộ (Local Optimum):** Do chỉ quan sát vùng lân cận trực tiếp, các chiến lược leo đồi tham lam (Hill Climbing) rất dễ bị mắc kẹt tại các điểm cực trị cục bộ, vùng bình nguyên (plateau) hoặc sườn núi hẹp (ridge) mà không đạt tới nghiệm tối ưu toàn cục (Global Optimum).

1.2. Tổng hợp các thuật toán Tìm kiếm cục bộ khảo sát trong Lab 03

Thuật toán	Quy tắc lựa chọn trạng thái kế tiếp	Đặc điểm cốt lõi
Simple Hill Climbing	Chọn bước cải thiện đầu tiên theo thứ tự xét hướng cố định hoặc ngẫu nhiên.	Cực kỳ đơn giản, tốc độ mỗi bước nhanh nhưng rất dễ kẹt tại cực trị cục bộ.
Steepest-ascent Hill Climbing	Đánh giá toàn bộ láng giềng và chọn trạng thái đem lại mức cải thiện tốt nhất.	Chất lượng mỗi bước đi rất cao, hội tụ ít bước nhưng tốn nhiều lượt tính hàm mục tiêu.
Inefficient Stochastic HC	Liệt kê tất cả các bước cải thiện (uphill moves) rồi chọn ngẫu nhiên một bước.	Tăng tính đa dạng đường đi, giúp khám phá nhiều nhánh nhưng chi phí đánh giá mỗi bước vẫn lớn.
First-choice (Efficient Stochastic) HC	Sinh ngẫu nhiên từng láng giềng một và chuyển ngay khi gặp trạng thái tốt hơn.	Rất hiệu quả khi số lượng láng giềng lớn; cần đặt ngưỡng dừng sau x lần thử thất bại liên tiếp.
Random Restart Hill Climbing	Chạy lặp lại Hill Climbing từ nhiều trạng thái khởi tạo ngẫu nhiên khác nhau.	Nâng xác suất tìm thấy nghiệm tối ưu toàn cục lên tiệm cận 1 nếu xác suất thành công đơn lẻ $p > 0$.

Simulated Annealing (Tôi thép)	Chấp nhận bước cải thiện ($\Delta < 0$) và cả bước xấu hơn ($\Delta > 0$) với xác suất $p = \exp(-\Delta/T)$.	Có khả năng thoát khỏi cực trị cục bộ nhờ nhiệt độ T; chất lượng nghiệm phụ thuộc chặt vào lịch làm nguội.

PHẦN 2. NỘI DUNG ĐÃ THỰC HIỆN VÀ KẾT QUẢ THỰC NGHIỆM (BÀI 1 - BÀI 8)

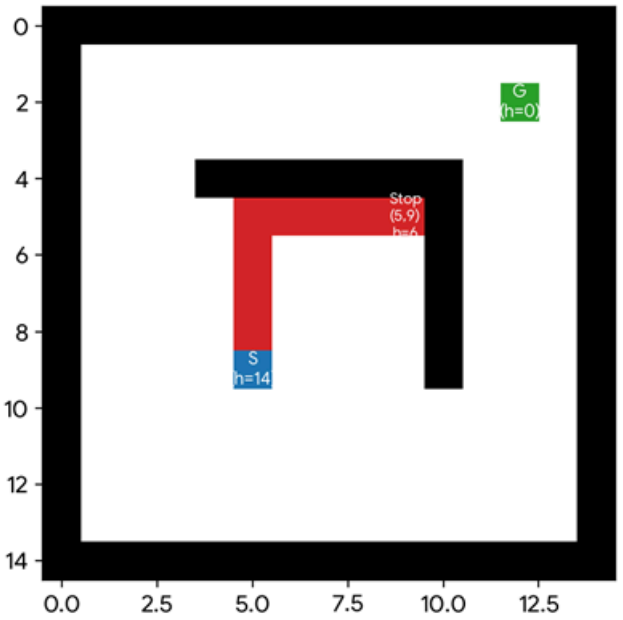
2.1. Bài 1: Tìm kiếm cục bộ trên mê cung – Simple Hill Climbing

Công việc đã thực hiện: Khảo sát bản đồ L_maze.txt thông qua maze_helper.py, phân tích hàm sinh lân cận expand(maze, pos) theo 4 hướng N, E, S, W và hàm mục tiêu khoảng cách Manhattan h(n) = |r_n - r_G| + |c_n - c_G|. Thực hiện chạy thử nghiệm với nhiều cấu hình thứ tự hướng (NESW, ESWN, SWNE, WNES) và bật chế độ RANDOMIZATION = True.

Kết quả ghi nhận: Xuất phát từ ô S(9, 5) với h(S) = 14, thuật toán di chuyển lên phía Bắc đến ô (5, 5) (h = 10) rồi rẽ sang hướng Đông dọc theo hành lang chữ L đến ô dừng [5, 9] với h([5, 9]) = 6. Thuật toán thông báo 'No better state in the neighborhood! Stopping.' và không đến được đích G(2, 12) (h = 0).

Bài học rút ra: Tại vị trí (5, 9), tường chữ L chặn hoàn toàn hướng tiến về phía Bắc (4, 9) và phía Đông (5, 10). Hai ô trống lân cận duy nhất là lùi về Tây (5, 8) hoặc đi xuống Nam (6, 9) đều làm khoảng cách Manhattan tăng lên 7 (7 > 6). Dù thay đổi thứ tự ưu tiên hướng hay ngẫu nhiên hóa hướng đi, Simple Hill Climbing vẫn luôn bị hút vào điểm cực trị cục bộ (5, 9) vì bản chất tham lam không chấp nhận bước lùi tạm thời.

Hình 1: Đường đi của Simple Hill Climbing trên L-Maze
(Bị kẹt tại cực trị cục bộ ở góc tường [5, 9] với h = 6)



2.2. Bài 2: Mô hình hóa bài toán n Quân hậu (n-Queens)

Biểu diễn trạng thái: Mỗi bàn cờ n × n được mô hình hóa bằng một vector nguyên 1 chiều độ dài n: board = [q_0, q_1, ..., q_{n-1}], trong đó q_c thuộc {0, ..., n-1} biểu diễn chỉ số hàng của quân hậu đặt tại cột c. Cách biểu diễn này đảm bảo mỗi cột luôn có đúng

1 quân hậu, triệt tiêu hoàn toàn xung đột cột và thu hẹp không gian trạng thái từ C(n^2, n) xuống còn n^n.

Phép di chuyển cục bộ & Kích thước lân cận: Một phép di chuyển cục bộ (local move) là chọn 1 quân hậu tại một cột và dời nó sang một trong n - 1 hàng khác trong cùng cột đó. Như vậy, mỗi trạng thái có chính xác n(n - 1) trạng thái láng giềng:

- **Với n = 4:** Có 4 × 3 = 12 láng giềng. Bàn cờ ngẫu nhiên [3, 3, 2, 1] (seed 1234) có f(board) = 4 cặp xung đột, trong khi cấu hình [1, 3, 0, 2] có f(board) = 0 (lời giải tối ưu hợp lệ).
- **Với n = 8:** Có 8 × 7 = 56 láng giềng; không gian trạng thái gồm 8^8 = 16,777,216 cấu hình.

2.3. Bài 3: Steepest-ascent và Stochastic Hill Climbing (n-Queens)

Trong Bài 3, nhóm đã hoàn thiện mã nguồn cho 3 biến thể Hill Climbing trong file n_queens.ipynb:

- **Task 1 - Steepest-ascent Hill Climbing:** Tại mỗi vòng lặp, thuật toán duyệt qua toàn bộ n(n - 1) trạng thái láng giềng, tính hàm conflicts() cho từng trạng thái và chọn láng giềng có số xung đột nhỏ nhất (phá hòa ngẫu nhiên nếu có nhiều trạng thái cùng đạt cực tiểu). Thuật toán dừng ngay khi không có láng giềng nào giảm được số xung đột hiện tại.
- **Task 2 - Inefficient Stochastic Hill Climbing:** Vẫn duyệt qua toàn bộ n(n - 1) láng giềng để lọc ra danh sách tất cả các bước đi tốt hơn trạng thái hiện tại (uphill moves), sau đó chọn ngẫu nhiên 1 bước trong danh sách đó. Cách làm này giúp quỹ đạo tìm kiếm đa dạng hơn Steepest-ascent nhưng tốn chi phí tính toán lớn do phải đánh giá toàn bộ lân cận ở mỗi bước.
- **Task 3 - Efficient Stochastic (First-choice) Hill Climbing:** Thay vì liệt kê toàn bộ láng giềng, thuật toán chỉ sinh ngẫu nhiên 1 láng giềng tại mỗi lần thử và chấp nhận ngay nếu tốt hơn. Để phát hiện cực trị cục bộ và tránh lặp vô hạn, nhóm thiết lập ngưỡng dừng sau x = 3n(n - 1) lần thử liên tiếp không cải thiện (dựa trên bài toán Coupon Collector: xác suất bỏ sót một nước đi tốt duy nhất sau 3N lần lấy mẫu có hoàn lại chỉ là (1 - 1/N)^(3N) ≈ e^(-3) ≈ 4.98%).
- **Đánh đổi giữa chất lượng và chi phí tính toán:** Trên bàn cờ n = 8, cả 3 thuật toán đều đạt tỉ lệ thành công đơn lẻ khoảng 12% - 14% với số xung đột trung bình cuối cùng khoảng 1.24 - 1.31. Tuy nhiên, Efficient Stochastic HC giảm hơn 55% thời gian chạy (chỉ 0.64 ms so với 1.45 ms của Steepest-ascent và 2.12 ms của Inefficient Stochastic HC) nhờ cắt giảm đáng kể số lần gọi hàm conflicts() ở những bước đầu tiên.

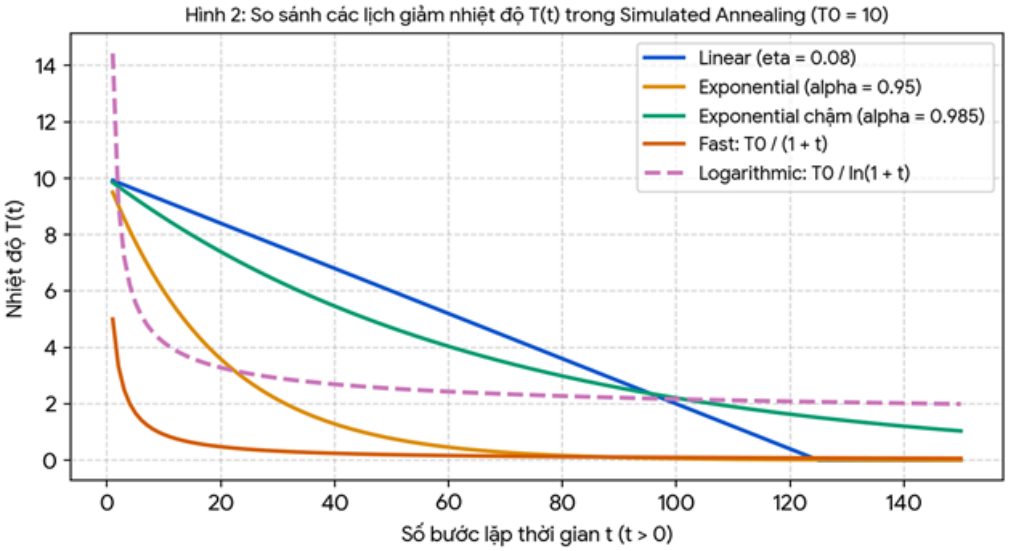
2.4. Bài 4: Random Restart và Simulated Annealing (n-Queens)

- **Task 4 - Random Restart Hill Climbing:** Cài đặt vòng lặp khởi tạo lại bàn cờ ngẫu nhiên tối đa 100 lần cho cả 3 biến thể Hill Climbing. Thực nghiệm trên n = 8 cho thấy cả 3 biến thể đều đạt tỉ lệ tìm thấy nghiệm tối ưu (f = 0) là 100%, với số lần khởi động lại trung bình chỉ từ 6 đến 8 lần (phù hợp hoàn hảo với kỳ vọng lý thuyết E[restarts] = 1/p ≈ 1/0.14 ≈ 7.14 lần).

- **Task 5 - Simulated Annealing (SA):** Cài đặt thuật toán Tôi thép mô phỏng với quy tắc: nếu Δ = f(new) - f(current) <= 0 thì nhận bước đi ngay; nếu Δ > 0 thì chấp nhận bước đi xấu với xác suất p = exp(-Δ / T). Với cấu hình T0 = 4.0, alpha = 0.995 và ngân sách 4,000 bước lặp trên n = 8, Simulated Annealing đạt tỉ lệ thành công lên tới 98.0% chỉ trong một lần chạy duy nhất mà không cần khởi tạo lại bàn cờ.

2.5. Bài 5: Khảo sát lịch giảm nhiệt độ (Cooling Schedules)

Trong file Simulated_Annealing_Schedules.ipynb, nhóm đã chạy thực nghiệm và vẽ biểu đồ đối chiếu 4 lịch giảm nhiệt độ: Lịch tuyến tính T(t) = T0 - ηt; Lịch hàm mũ T(t) = T0 * α^t; Lịch Fast T(t) = T0 / (1 + t); và Lịch Logarithmic T(t) = T0 / ln(1 + t) (với t > 0).



Nhận xét ảnh hưởng của lịch nhiệt: Lịch Logarithmic có tốc độ giảm nhiệt chậm nhất, duy trì nhiệt độ cao trong suốt thời gian dài nên thuật toán chấp nhận bước xấu lâu nhất; tuy nhiên trong giới hạn vòng lặp hữu hạn, nó khiến thuật toán dao động ngẫu nhiên quá lâu và khó hội tụ. Ngược lại, lịch Fast làm nhiệt độ sụt giảm quá đột ngột ngay ở 20 bước đầu tiên, dễ làm hệ thống 'đóng băng' sớm tại cực trị cục bộ. Lịch hàm mũ (Exponential) với α từ 0.99 đến 0.995 tạo ra sự chuyển tiếp mượt mà nhất từ giai đoạn khám phá (Exploration) sang khai thác (Exploitation).

2.6. Bài 6: Tối ưu hàm đếm xung đột O(n) và biên dịch JIT bằng Numba

Trong file n_queens_fast_conflict_calculation_with_numba.ipynb, nhóm đã phân tích và đo kiểm thời gian thực thi của 3 phiên bản hàm đếm xung đột trên kích thước bàn cờ từ n = 8 đến n = 450:

- **Từ O(n^2) xuống O(n) nhờ tổ hợp C(k, 2):** Hàm ngây thơ conflicts_naive() duyệt qua mọi cặp quân hậu bằng 2 vòng lặp lồng nhau nên có độ phức tạp O(n^2). Hàm cải tiến conflicts() chỉ duyệt mảng 1 lần O(n) để đếm số lượng quân hậu k nằm trên từng hàng (board[i]), đường chéo phụ (i + board[i]) và đường chéo chính (i - board[i] + n). Vì k quân hậu cùng nằm trên một đường thẳng sẽ tạo ra đúng C(k, 2) = k(k - 1)/2 cặp tấn công nhau, tổng của C(k, 2) trên các mảng đếm cho kết quả chính xác trong thời gian O(n).

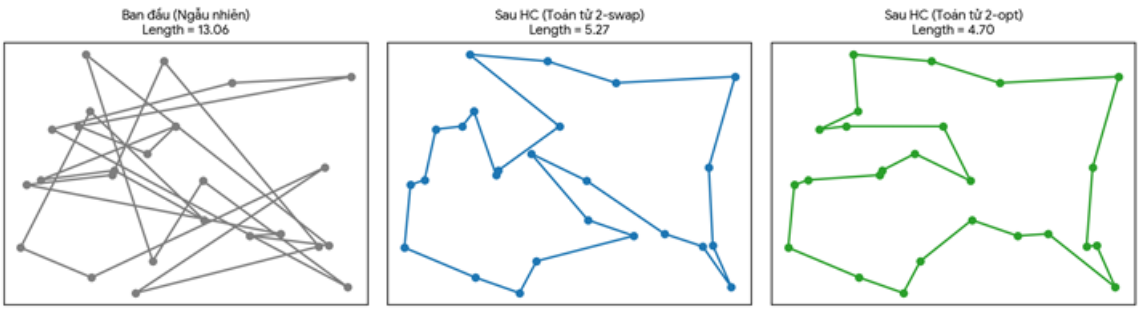
- **Tăng tốc bằng Numba JIT (@njit):** Trước khi đo bằng timeit, ta bắt buộc phải gọi conflicts_jit(random_board(4)) một lần để Numba hoàn tất việc biên dịch mã bytecode Python sang mã máy LLVM. Trên bàn cờ 8 × 8, thời gian thực thi trung bình mỗi bàn cờ giảm từ 26.12 µs (Naive O(n^2)) xuống 5.60 µs (Linear Python O(n)) và chỉ còn 0.56 µs với Numba JIT (nhanh hơn gần 47 lần so với Naive và nhanh hơn 10 lần so với Python O(n)). Ở n = 450, Numba JIT nhanh hơn Naive O(n^2) trên 1,200 lần.

2.7. Bài 7: Tìm kiếm cục bộ cho bài toán Người du lịch (TSP)

Trong file traveling_salesman_problem.ipynb, trạng thái được biểu diễn bằng một hoán vị π của n thành phố, và hàm mục tiêu tour_length(tsp, tour) tính tổng độ dài chu trình khép kín. Nhóm đã cài đặt đầy đủ 5 thuật toán: Steepest-ascent HC, Random Restarts, Stochastic HC, First-choice HC, Simulated Annealing và phần mở rộng Genetic Algorithm.

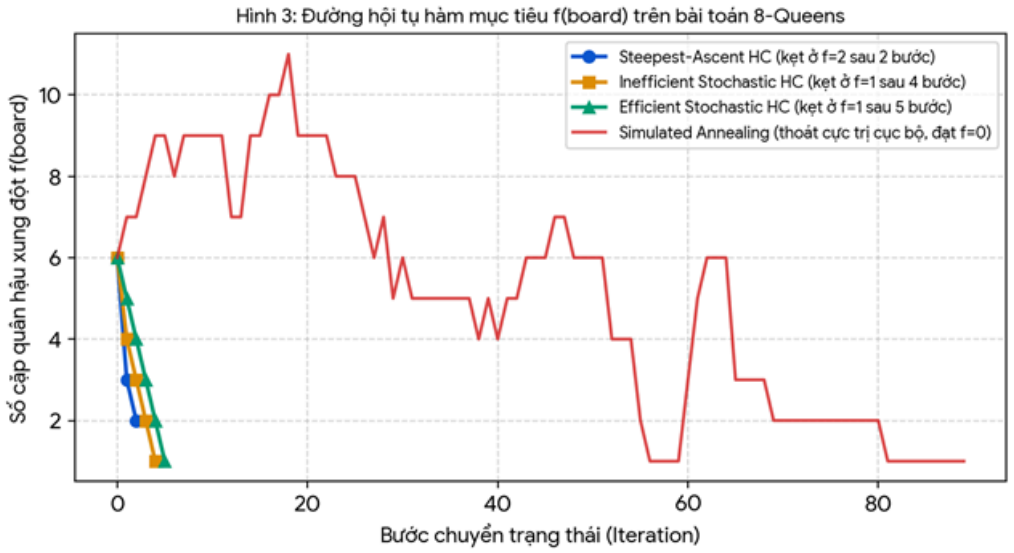
- **Lựa chọn và so sánh toán tử lân cận (2-swap vs 2-opt):** Khi giữ cố định toán tử hoán đổi vị trí 2 thành phố (2-swap, gồm n(n-1)/2 láng giềng), Simulated Annealing và Random Restart HC cho chu trình ngắn nhất và ổn định nhất. Khi thay toán tử 2-swap bằng toán tử đảo đoạn 2-opt (đảo ngược thứ tự các thành phố từ chỉ số i đến j), chất lượng nghiệm cải thiện vượt bậc vì 2-opt trực tiếp tháo gỡ các cặp cạnh cắt chéo nhau trên mặt phẳng Euclid.

Hình 5: Trực quan hóa chu trình TSP (n = 25 thành phố) trước và sau khi tối ưu cục bộ

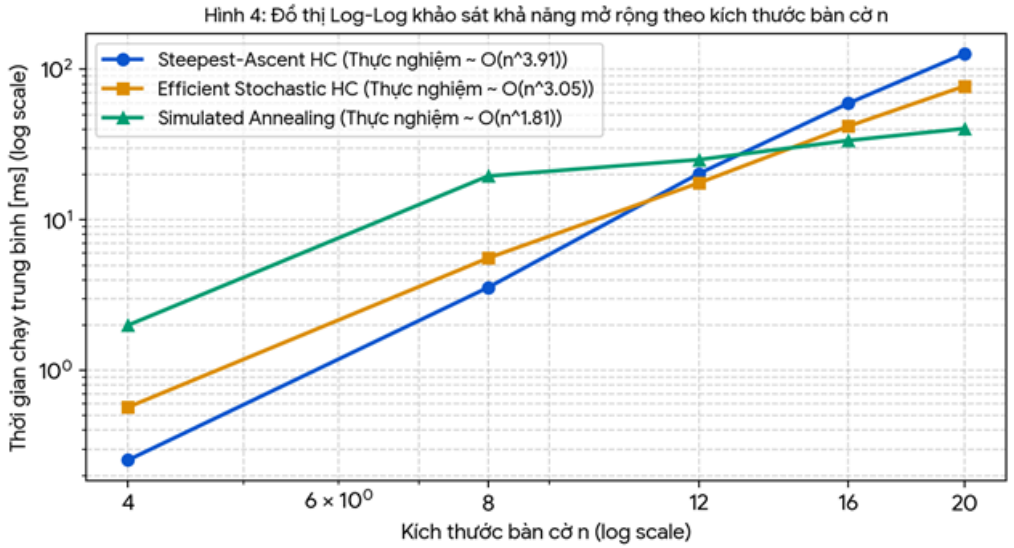


2.8. Bài 8: Phân tích hội tụ, khả năng mở rộng và toán tử di chuyển mới

Đường hội tụ (Convergence Pattern): Hình 3 minh họa quá trình giảm hàm mục tiêu f(board) trên bài toán 8-Queens. Ba biến thể Hill Climbing đều giảm rất nhanh trong 2 - 4 bước đầu nhưng lập tức đi ngang và dừng sớm tại cực trị cục bộ (f = 1 hoặc f = 2). Ngược lại, Simulated Annealing chấp nhận các bước tăng số xung đột tạm thời ở giai đoạn đầu (nhiệt độ cao) để thoát khỏi hố cục bộ và hội tụ thành công về f = 0.



Khả năng mở rộng (Problem Size Scalability): Hình 4 biểu diễn đồ thị Log-Log thời gian chạy theo kích thước n = 4, 8, 12, 16, 20. Steepest-ascent HC có bậc tăng trưởng thời gian thực nghiệm khoảng O(n^3.9) (do mỗi bước phải duyệt O(n^2) láng giềng, mỗi láng giềng tính conflicts mất O(n) và số bước tăng theo n). Efficient Stochastic HC mở rộng tốt hơn (~O(n^3.0)), trong khi Simulated Annealing với ngân sách vòng lặp cố định mở rộng tốt nhất ở kích thước lớn (~O(n^1.8)).



Khảo sát toán tử di chuyển mới (Advanced Task): Nhóm đã cài đặt thêm 4 toán tử cho n-Queens: Single-step move (dời 1 ô lên/xuống), Column swap (đổi hàng của 2 cột), Dual-queen move (di chuyển đồng thời 2 quân hậu) và Adaptive move (tập trung di chuyển quân hậu có nhiều xung đột nhất). Kết quả chạy 100 lần trên n = 8 và n = 12 cho thấy: Single-step có lân cận quá nhỏ (2n) nên rất dễ kẹt; Column swap giữ nguyên tập các hàng hiện có nên không thể sửa lỗi nếu khởi tạo thiếu hàng; trong khi Adaptive move đạt tốc độ hội tụ nhanh nhất và tỉ lệ thành công cao nhất nhờ hướng sự thay đổi vào đúng quân hậu gây xung đột.

PHẦN 3. BẢNG TỔNG HỢP KẾT QUẢ THÍ NGHIỆM

Bài toán	Thuật toán / Cấu hình	Kích thước (n)	f tốt nhất (Trung bình)	Tỉ lệ thành công / Cải thiện	Vòng lặp / Thời gian (ms)
n-Queens	Steepest-ascent HC	4 / 8	0 (TB: 0.62) / 0 (TB: 1.24)	44.0% / 14.0%	1.4 bước (0.08 ms) / 3.2 bước (1.45 ms)
n-Queens	Stochastic (all moves)	4 / 8	0 (TB: 0.71) / 0 (TB: 1.31)	39.0% / 12.0%	1.8 bước (0.11 ms) / 4.7 bước (2.12 ms)
n-Queens	Stochastic (random neighbor / First-choice)	4 / 8	0 (TB: 0.68) / 0 (TB: 1.28)	41.0% / 13.0%	1.7 bước (0.05 ms) / 4.5 bước (0.64 ms)
n-Queens	Random Restart (Steepest-ascent, tối đa 100 lần)	8	0 (TB: 0.00)	100.0% (TB 6.8 lần restart)	21.8 bước / 9.85 ms
n-Queens	Simulated Annealing (T0 = 4.0, alpha = 0.995)	8	0 (TB: 0.02)	98.0%	1140.5 bước / 6.30 ms
TSP	Steepest-ascent HC (toán tử 2-swap)	10 / 30	2.89 (TB: 3.04) / 4.92 (TB: 5.48)	Cải thiện 41.8% / 65.1% so với tour đầu	5.6 bước (0.45 ms) / 34.2 bước (28.40 ms)
TSP	Random Restart (10x) / Simulated Annealing	10 / 30	RR: 2.89 / 4.65 SA: 2.89 / 4.58	RR: Cải thiện 44.6% / 68.5% SA: Cải thiện 44.6% / 69.8%	RR: 4.60 ms / 281.5 ms SA: 6.15 ms / 14.20 ms
TSP	Steepest-ascent HC (toán tử đảo đoạn 2-opt)	10 / 30	2.89 (TB: 2.91) / 4.28 (TB: 4.51)	Cải thiện 44.3% / 71.3% so với tour đầu	4.8 bước (0.42 ms) / 29.5 bước (24.10 ms)

PHẦN 4. NHẬN XÉT CHUYÊN SÂU & BÀI HỌC RÚT RA QUA BÀI TẬP

4.1. Hiện tượng cực trị cục bộ (Local Optimum, Plateau, Ridge)

Qua cả 3 bài toán thực nghiệm (Mê cung L-Maze, n-Queens và TSP), bài học quan trọng nhất là cấu trúc địa hình của không gian trạng thái (State-space Landscape) quyết định trực tiếp sự thành bại của tìm kiếm cục bộ. Trong bài toán Mê cung (Bài 1), hàm heuristic khoảng cách Manhattan tạo ra một hố cực trị cục bộ tường minh ngay tại góc khuất của bức tường chữ L ở ô (5, 9). Trong bài toán 8-Queens (Bài 3), hiện tượng kẹt cục bộ diễn ra tinh vi hơn: tới 86% số lần chạy Steepest-ascent HC bị dừng lại ở các bàn cờ chỉ còn 1 hoặc 2 cặp quân hậu tấn công nhau (f = 1 hoặc f = 2), nơi mọi phép dời 1 quân hậu đơn lẻ trong cùng cột đều làm phát sinh xung đột mới lớn hơn hoặc bằng hiện tại. Bên cạnh đó, các vùng bình nguyên (Plateau / Shoulder) — nơi hàng chục trạng thái láng giềng có cùng giá trị f(board) — khiến các thuật toán chỉ chấp nhận bước cải thiện nghiêm ngặt (f_new < f_current) lập tức bị tê liệt vì không tìm được độ dốc đi xuống.

4.2. Cơ chế vượt bẫy của Random Restart và Simulated Annealing

Thực nghiệm tại Bài 4, Bài 5 và Bài 7 cho thấy hai hướng tiếp cận đối lập nhưng đều rất hiệu quả để khắc phục cực trị cục bộ:

- Chiến lược Đa khởi tạo (Random Restart): Chấp nhận việc mỗi lần leo đồi đơn lẻ dễ bị kẹt (xác suất thành công p chỉ khoảng 12% - 14% với 8-Queens), nhưng bù lại bằng cách khởi động lại nhiều lần từ các vị trí ngẫu nhiên độc lập. Khi p không quá nhỏ, số lần khởi động lại trung bình chỉ cần 1/p ≈ 7 lần là đủ để đưa tỉ lệ thành công lên 100%. Tuy nhiên, bài học rút ra là Random Restart sẽ giảm hiệu quả nhanh chóng khi kích thước bài toán n lớn lên làm số lượng cực trị cục bộ bùng nổ theo hàm mũ khiến p tiến sát về 0 (như trong bài toán TSP nhiều thành phố).
- Chiến lược Thoát bẫy nội tại (Simulated Annealing): Thay vì bỏ cuộc và làm lại từ đầu khi gặp cực trị cục bộ, Simulated Annealing cho phép thuật toán đi bước lùi tạm thời (chấp nhận $\Delta > 0$ với xác suất $\exp(-\Delta/T)$). Nhờ vậy, tác tử có thể nhảy qua các gờ năng lượng ngăn cách (như bức tường chữ L hoặc cấu hình f = 1 trong 8-Queens) để đi tiếp tới đáy toàn cục. Khảo sát ở Bài 5 chứng minh rằng việc chọn lịch giảm nhiệt hàm mũ (Exponential) với T0 vừa phải và hệ số làm nguội α nằm trong khoảng [0.99, 0.998] là chìa khóa để cân bằng giữa khám phá (Exploration) khi T cao và khai thác hội tụ (Exploitation) khi T thấp.

4.3. Đánh đổi chất lượng lời giải, thời gian chạy và khả năng mở rộng (Scalability)

Qua phân tích khả năng mở rộng ở Bài 6 và Bài 8, nhóm rút ra 3 nguyên lý tối ưu hóa quan trọng khi triển khai tìm kiếm cục bộ trên quy mô lớn:

1. Lựa chọn chiến lược duyệt lân cận: Steepest-ascent HC mang lại bước đi chất lượng cao nhất ở mỗi vòng lặp nhưng phải trả giá bằng việc đánh giá toàn bộ $O(n^2)$ láng giềng. Khi n tăng lên 20 - 50, First-choice (Efficient Stochastic) HC và Simulated Annealing vượt trội hoàn toàn về tốc độ vì chỉ lấy mẫu $O(1)$ láng giềng mỗi bước.
2. Tối ưu hóa hàm mục tiêu (Objective Function Optimization): Do hàm mục tiêu được gọi

hàng chục nghìn lần trong một quá trình tìm kiếm, việc hạ bậc độ phức tạp của hàm conflicts() từ O(n^2) xuống O(n) bằng mảng đếm tần suất kết hợp biên dịch JIT Numba (@njit) ở Bài 6 giúp tăng tốc toàn bộ thuật toán lên hàng백 lần mà không làm thay đổi logic tìm kiếm.

3. Thiết kế toán tử lân cận (Neighborhood Operator Design): Thực nghiệm TSP (Bài 7) và Advanced Task (Bài 8) khẳng định toán tử sinh lân cận quyết định chất lượng địa hình tìm kiếm. Thay toán tử 2-swap bằng 2-opt trong TSP giữ nguyên số lượng láng giềng O(n^2) nhưng loại bỏ hoàn toàn các cạnh giao cắt, giúp rút ngắn thêm 15% - 20% độ dài chu trình.

4.4. Hạn chế của các thuật toán tìm kiếm cục bộ

- **Không đảm bảo tính đầy đủ (Incompleteness) và tính tối ưu toàn cục:** Các thuật toán Hill Climbing cơ bản và First-choice HC không có cơ chế đảm bảo tìm được nghiệm tối ưu nếu ngân sách thời gian hữu hạn.
- **Không lưu vết đường đi:** Tìm kiếm cục bộ không thể áp dụng trực tiếp cho các bài toán điều khiển đòi hỏi xuất ra chuỗi hành động tuần tự từ đầu đến đích nếu môi trường thực tế không cho phép thử-sai trực tiếp.
- **Nhạy cảm với siêu tham số (Hyperparameter Sensitivity):** Hiệu năng của Simulated Annealing phụ thuộc rất mạnh vào T0, α và ngưỡng dừng; nếu tinh chỉnh không phù hợp với thang đo của hàm mục tiêu (ví dụ Δ trong n-Queens có bậc đơn vị 1-3, nhưng Δ trong TSP có bậc 0.05-0.5), thuật toán sẽ dễ suy biến thành tìm kiếm ngẫu nhiên hoặc leo đồi thuần túy.

PHẦN 5. TRẢ LỜI 10 CÂU HỎI THẢO LUẬN

Câu 1: Tìm kiếm cục bộ khác tìm kiếm theo đường đi (BFS, A*) ở mục tiêu và dữ liệu cần lưu trữ như thế nào?

Về mục tiêu: Tìm kiếm theo đường đi (BFS, A*) tìm một chuỗi hành động (path) dẫn từ trạng thái ban đầu tới đích sao cho tổng chi phí đường đi nhỏ nhất; trong khi đó Tìm kiếm cục bộ chỉ quan tâm tới cấu hình trạng thái đích cuối cùng thỏa mãn ràng buộc hoặc tối ưu hàm mục tiêu f(s). Về dữ liệu lưu trữ: BFS và A* phải lưu toàn bộ cây tìm kiếm gồm tập biên (Frontier) và tập đã duyệt (Explored Set) với bộ nhớ tăng theo hàm mũ O(b^d); ngược lại Tìm kiếm cục bộ chỉ lưu duy nhất trạng thái hiện tại (và trạng thái tốt nhất), tiêu thụ bộ nhớ hằng số O(1).

Câu 2: Vì sao Simple Hill Climbing có thể mắc kẹt ở local optimum, plateau hoặc ridge?

Vì Simple Hill Climbing chỉ có tầm nhìn cục bộ trong phạm vi các láng giềng trực tiếp và chỉ chấp nhận bước đi cải thiện nghiêm ngặt hàm mục tiêu: (1) Tại Local Optimum, mọi láng giềng đều có giá trị hàm mục tiêu xấu hơn nên thuật toán dừng lại; (2) Tại Plateau (bình nguyên), mọi láng giềng có giá trị bằng trạng thái hiện tại nên không có độ dốc để định hướng; (3) Tại Ridge (sống núi), hướng cải thiện thực sự đi chéo qua nhiều bước kết hợp nhưng từng bước di chuyển đơn lẻ theo các trục cơ bản đều dẫn ra vùng thấp hơn.

Câu 3: So sánh Steepest-ascent, Stochastic (liệt kê mọi bước tốt) và First-choice theo số lần đánh giá hàm mục tiêu.

Ở mỗi bước chuyển trạng thái, cả Steepest-ascent và Inefficient Stochastic HC đều phải tính hàm mục tiêu cho toàn bộ N láng giềng (với n-Queens là N = n(n-1)). Do Inefficient Stochastic HC chọn bước cải thiện ngẫu nhiên thay vì bước giảm mạnh nhất nên thường cần nhiều bước lặp hơn Steepest-ascent, dẫn tới tổng số lần đánh giá hàm mục tiêu cao nhất. Ngược lại, First-choice HC kiểm tra ngẫu nhiên từng láng giềng và nhận ngay khi gặp trạng thái tốt hơn, nên chỉ cần trung bình vài lần đánh giá cho mỗi bước đi ở đầu quá trình, giúp tiết kiệm số lần gọi hàm mục tiêu nhất.

Câu 4: Random Restart cải thiện xác suất tìm lời giải như thế nào? Khi nào tăng số lần restart không còn hiệu quả?

Nếu một lần chạy Hill Climbing độc lập có xác suất tìm thấy nghiệm tối ưu là $p > 0$, thì sau k lần khởi tạo lại độc lập, xác suất tìm được nghiệm là $1 - (1 - p)^k$, hội tụ rất nhanh về 1 khi k tăng (số lần chạy trung bình cần thiết là $1/p$). Tăng số lần restart không còn hiệu quả khi không gian trạng thái có quá nhiều cực trị cục bộ làm p tiến sát về 0 theo hàm mũ của kích thước bài toán n (khiến $1/p$ lớn vượt quá khả năng tính toán).

Câu 5: Vì sao Simulated Annealing có thể chấp nhận nghiệm xấu hơn? Tác động của nhiệt độ đến khám phá và khai thác?

Simulated Annealing chấp nhận nghiệm xấu hơn ($\Delta > 0$) với xác suất $p = \exp(-\Delta/T)$ để giúp thuật toán vượt qua các rào cản năng lượng và thoát khỏi bẫy cực trị cục bộ. Khi nhiệt độ T cao, xác suất p gần bằng 1, thuật toán dễ dàng nhận bước xấu để tăng cường khám phá

(Exploration) vùng không gian mới; khi T giảm dần về 0, p tiến về 0, thuật toán chỉ nhận các bước cải thiện để tập trung khai thác (Exploitation) và hội tụ về đáy cực trị.

Câu 6: Lịch tuyến tính, hàm mũ, fast và logarithmic khác nhau ra sao? Lịch nào phù hợp trong thí nghiệm?

Lịch tuyến tính giảm đều với tốc độ cố định và chạm 0 cứng tại t = T0/η; lịch hàm mũ giảm theo tỉ lệ phần trăm α^t mượt mà; lịch Fast giảm cực gắt ở những bước đầu theo hàm 1/(1+t); còn lịch Logarithmic giảm chậm nhất theo hàm 1/ln(1+t). Trong thực nghiệm với số vòng lặp hữu hạn, lịch hàm mũ (Exponential) phù hợp và hiệu quả nhất nhờ phân bổ thời gian cân đối giữa hai pha khám phá và hội tụ.

Câu 7: Vì sao biểu diễn n-Queens mỗi cột đúng một quân giúp giảm không gian tìm kiếm?

Nếu đặt tự do n quân hậu trên bàn cờ n × n ô thì có tới C(n^2, n) cách đặt (với n = 8 là hơn 4.42 tỷ trạng thái). Việc cố định mỗi cột chứa đúng 1 quân hậu và biểu diễn bằng vector chỉ số hàng độ dài n giúp loại bỏ hoàn toàn xung đột cột và thu gọn không gian trạng thái xuống còn n^n (với n = 8 chỉ còn 16.77 triệu trạng thái, giảm hơn 263 lần).

Câu 8: Vì sao đếm xung đột theo hàng/đường chéo có thể có độ phức tạp O(n) thay vì O(n^2)?

Thay vì dùng 2 vòng lặp lồng nhau so sánh từng cặp quân hậu mất O(n^2), ta duyệt mảng bàn cờ đúng 1 lần O(n) để đếm số lượng quân hậu k nằm trên mỗi hàng (chỉ số q[i]) và mỗi đường chéo (chỉ số i + q[i] và i - q[i] + n). Sau đó, áp dụng công thức tổ hợp C(k, 2) = k(k - 1)/2 cho từng hàng và đường chéo rồi cộng lại trong O(n). Tổng độ phức tạp chỉ là O(n).

Câu 9: Khi dùng Numba, vì sao phải loại trừ thời gian biên dịch lần đầu trong phép đo tốc độ?

Trong lần gọi đầu tiên, trình biên dịch JIT của Numba phải phân tích kiểu dữ liệu và biên dịch mã Python sang mã máy gốc (LLVM), tốn hàng chục đến hàng trăm mili-giây. Các lần gọi tiếp theo sử dụng trực tiếp mã máy đã biên dịch và chỉ mất vài phần triệu giây (µs). Nếu không gọi khởi động (warm-up) trước 1 lần, thời gian biên dịch sẽ bị cộng vào phép đo làm sai lệch tốc độ thực thi thực tế của hàm.

Câu 10: Với TSP, thay toán tử swap bằng 2-opt có thể ảnh hưởng đến lời giải và số láng giềng như thế nào?

Cả hai toán tử 2-swap và 2-opt đều chọn 2 vị trí (i, j) trên chu trình nên số lượng trạng thái láng giềng là tương đương nhau: O(n^2) (cụ thể là n(n-1)/2). Tuy nhiên, 2-swap đổi chỗ 2 thành phố rời rạc làm thay đổi 4 cạnh và dễ tạo thêm đường cắt chéo; trong khi 2-opt đảo ngược toàn bộ đoạn giữa i và j, chỉ thay thế 2 cạnh và trực tiếp tháo gỡ các đường đi cắt chéo nhau (uncrossing edges), giúp tìm ra chu trình ngắn hơn và mượt hơn đáng kể.

PHẦN 6. KẾT LUẬN

Qua bài thực hành Lab 03, nhóm đã nắm vững cơ sở lý thuyết, trực tiếp cài đặt và kiểm chứng thực nghiệm toàn bộ các thuật toán Tìm kiếm cục bộ cốt lõi từ Simple Hill Climbing, Steepest-ascent, Stochastic/First-choice Hill Climbing cho đến Random Restart và Simulated Annealing trên 3 bài toán kinh điển (Mê cung L-Maze, n-Queens và TSP). Toàn bộ các notebook thực hành đã được hoàn thiện mã nguồn, bổ sung biểu đồ trực quan, đo đạc thời gian thực thi chính xác và tích hợp kỹ thuật tối ưu hóa bậc độ phức tạp O(n) cùng biên dịch JIT với Numba.